Pyth Lazer Contracts

Security Assessment

February 16th, 2026 — Prepared by OtterSec

Michał Bochnak embe221ed@osec.io

Sangsoo Kang sangsoo@osec.io

Andreas Mantzoutas andreas@osec.io

Thiago Tavares thitav@osec.io

Table of Contents

Appendices

01 — Executive Summary

Overview

Pyth engaged OtterSec to assess the `pyth-lazer` program. This assessment was conducted between January 28th and February 9th, 2026. For more information on our auditing methodology, refer to Appendix B.

Key Findings

We produced 1 finding throughout this audit engagement.

We suggested refinements to address minor inconsistencies in documentation and structure, improving overall clarity and maintainability (OS-PLC-SUG-00).

Scope

The source code was delivered to us in a git repository at https://github.com/pyth-network/pyth-crosschain. This audit was performed against commit d6dd784.

A brief description of the program is as follows:

Name	Description
pyth-lazer	Sui package that enables users to seamlessly parse and verify cryptographically signed price feed data from the Pyth Network's high-frequency Lazer protocol.

02 — General Findings

Here, we present a discussion of general findings during our audit. While these findings do not present an immediate security impact, they represent anti-patterns and may result in security issues in the future.

ID	Description
OS-PLC-SUG-00	Recommendations for updating the codebase to ensure adherence to best coding practices.

Code Maturity

OS-PLC-SUG-00

Description

1. There is a mismatch in the module number in the text describing the `EMismatchedModule` error. The module number should be 3 instead of 2.

>_ lazer/contracts/sui/sources/governance.move RUST

```
#[error]
const EMismatchedModule: vector<u8> = "Mismatched governance header module number, should
    ↪  be 2";
```

2. Unpack the VAA directly inside `governance::process_incoming` to centralize the validation logic in a single location.
3. `i16::parse_magnitude` operates strictly on 16-bit two's-complement values, but accepts a `u64`, potentially allowing out-of-range inputs. Thus, it will be appropriate to update the `from` parameter type to `u16`.

>_ lazer/contracts/sui/sources/i16.move RUST

```
fun parse_magnitude(from: u64, negative: bool): u64 {
    // If positive, then return the input verbatim
    if (!negative) {
        return from
    };

    // Otherwise convert from two's complement by inverting and adding 1
    // For 16-bit numbers, we only invert the lower 16 bits
    let inverted = from ^ 0xFFFF;
    inverted + 1
}
```

Remediation

Implement the above-mentioned suggestions.

Patch

1. Issue #1 resolved in 310bb60.
2. Issue #2 resolved in 310bb60.
3. Issue #3 resolved in 3d951eb.

A — Vulnerability Rating Scale

We rated our findings according to the following scale. Vulnerabilities have immediate security implications. Informational findings may be found in the General Findings.

CRITICAL

Vulnerabilities that immediately result in a loss of user funds with minimal preconditions.

Examples:

- Misconfigured authority or access control validation.
- Improperly designed economic incentives leading to loss of funds.

HIGH

Vulnerabilities that may result in a loss of user funds but are potentially difficult to exploit.

Examples:

- Loss of funds requiring specific victim interactions.
- Exploitation involving high capital requirement with respect to payout.

MEDIUM

Vulnerabilities that may result in denial of service scenarios or degraded usability.

Examples:

- Computational limit exhaustion through malicious input.
- Forced exceptions in the normal user flow.

LOW

Low probability vulnerabilities, which are still exploitable but require extenuating circumstances or undue risk.

Examples:

- Oracle manipulation with large capital requirements and multiple transactions.

INFO

Best practices to mitigate future security risks. These are classified as general findings.

Examples:

- Explicit assertion of critical internal invariants.
- Improved input validation.

B — Procedure

As part of our standard auditing procedure, we split our analysis into two main sections: design and implementation.

When auditing the design of a program, we aim to ensure that the overall economic architecture is sound in the context of an on-chain program. In other words, there is no way to steal funds or deny service, ignoring any chain-specific quirks. This usually requires a deep understanding of the program's internal interactions, potential game theory implications, and general on-chain execution primitives.

One example of a design vulnerability would be an on-chain oracle that could be manipulated by flash loans or large deposits. Such a design would generally be unsound regardless of which chain the oracle is deployed on.

On the other hand, auditing the program's implementation requires a deep understanding of the chain's execution model. While this varies from chain to chain, some common implementation vulnerabilities include reentrancy, account ownership issues, arithmetic overflows, and rounding bugs.

As a general rule of thumb, implementation vulnerabilities tend to be more "checklist" style. In contrast, design vulnerabilities require a strong understanding of the underlying system and the various interactions: both with the user and cross-program.

As we approach any new target, we strive to comprehensively understand the program first. In our audits, we always approach targets with a team of auditors. This allows us to share thoughts and collaborate, picking up on details that others may have missed.

While sometimes the line between design and implementation can be blurry, we hope this gives some insight into our auditing procedure and thought process.